

HMI Copilot

“Build the whole plant: an overview plus a screen for each area, with pump status, flows, levels and alarms”

11

screens

560

objects

232

alarms

1,248

tags bound

0

by hand

And the file opens in EcoStruxure Operator Terminal Expert 4.4.

It was never saved by the product.

Today this is done by hand, every time

The cost the brief names	What it looks like on a real project
Manual screen development	Hand-place every lamp, bargraph and trend. Then repeat it for 40 near-identical equipment screens.
Expert-driven configuration	Only senior engineers know which object, animation and colour standard applies. Juniors block on them.
Complex tag integration	Hundreds of PLC symbols mapped by hand onto object properties, one property sheet at a time.
Inconsistency	Two engineers on one project produce two differently-named, differently-coloured HMIs.

A wrong binding is not a drawing error. It is pump 2's flow under pump 1's label, found during an upset.

A tag export and a sentence become a finished project

```
PLC tag export (.csv / .xlsx / symbol file) ---+
|
"Two pump station with running lamps,      ---+---> HMI COPILOT ---> project.eote
a flow display and a high-level alarm"      |           |           validation report
|                                           |
Company standards (naming, colours, layout) ----+         v
                                           LIVE CANVAS + BINDING MAP
                                           (the engineer sees it and steers it)
```

One file. File ► Open Project, and the HMI is there — screens drawn, tags declared, alarms configured, bindings wired. Nothing to import, nothing to assemble.

The engineer stays in control throughout: the AI proposes, the canvas makes the proposal legible, and a human accepts, edits or redirects. It never deploys anything itself.

Why this is possible at all

```
<project>.eote          ZIP    (nested entries use BACKSLASH separators)
|-- Project.dat, Target.dat  JSON  project identity, panel model, resolution
|-- Variables.db           SQLite the tags
|-- Alarm.db               SQLite alarm groups and alarms
|-- Recipe.db, Security.db ... SQLite other subsystems, passed through untouched
|-- Bindings.dat           JSON    Sources[] -> Bindings[] -> Targets[] graph
|-- Screens\Hierarchy.dat   JSON    screen order
+-- Screens\<guid>\Screen.dat JSON    the object tree
```

Plain JSON and plain SQLite. No signature. No encryption. Nothing to defeat.

Bindings are declarative — wiring a tag to a display is one JSON object. And the geometry is absolute Location, Width and Height inside a ViewBox, which maps one-to-one onto SVG. That last property is what lets one model drive both the preview and the file.

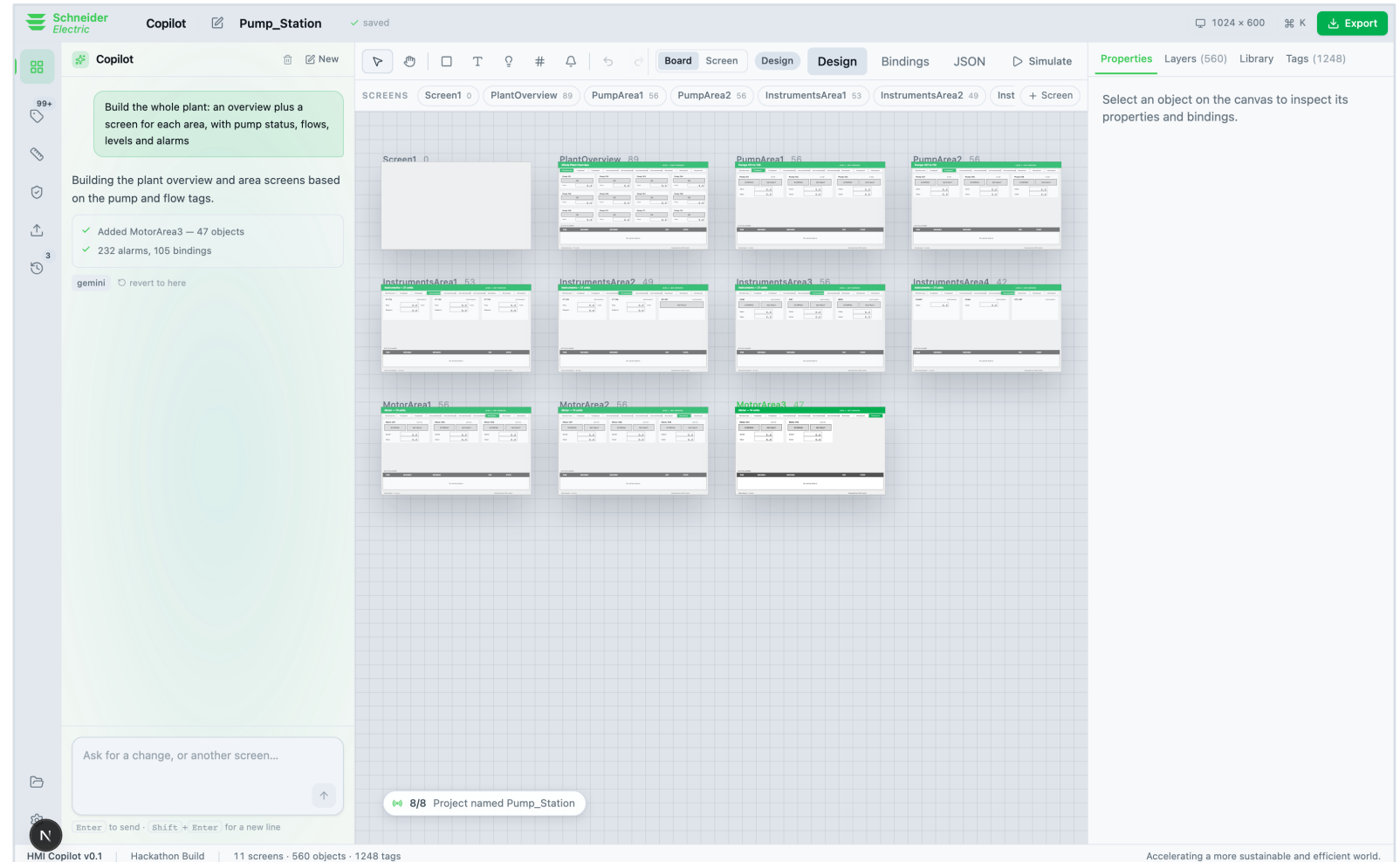
So the first thing we did was not design a UI. It was to prove a project could be written from scratch.

One sentence becomes a plant

DEMO · 1 · THE RESULT

Eleven screens on a board. An overview, then a screen per area, with navigation between them.

A 1,248-tag plant is not one screen — it is a hierarchy, and inferring that hierarchy from tag names is the actual problem. Every unit gets a screen.

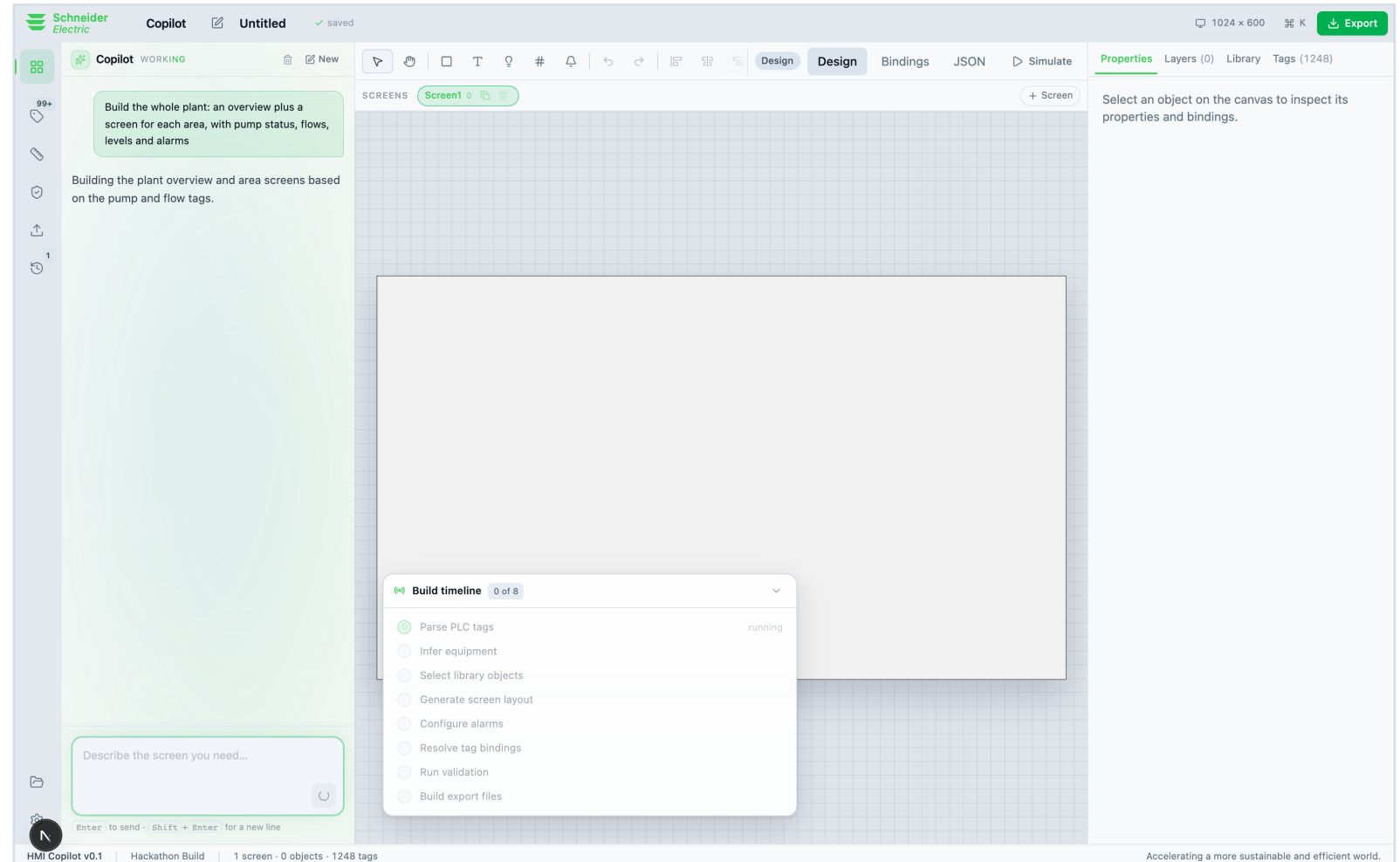


You watch it work, in engineering language

DEMO · 2 · THE PIPELINE,

Eight steps, each reporting what it produced, each clickable to select the objects it created.

Not “Thinking...”. Parsed 1,248 tags. Inferred pumps, instruments, motors. Configured 232 alarms. An engineer will not accept a screen they did not see being built.



Every tag, wired and visible

DEMO · 3 · THE PAIN POINT,

The binding map, drawn from the real Sources / Targets graph inside the .eote.

Tag integration is the pain point the brief names, and today it is invisible until commissioning. Here you can check it before you export.

The screenshot displays the Schneider Electric HMI Copilot interface for a project named "Pump_Station_Demo". The interface is divided into several sections:

- Left Sidebar:** Contains instructions for creating screens and adding elements. It also includes a "WHAT A REQUEST NEEDS" section with a list of requirements: "What the screen is about", "PLC tags to bind to", "Panel size", and "Somewhere to put it".
- Central Workspace:** Shows the "Bindings" tab with a search bar and a list of "PLC TAGS" and "OBJECT PROPERTIES". The "PLC TAGS" list includes: PMP_101_RUN, PMP_101_FLT, PMP_102_RUN, PMP_102_FLT, FT_101_PV, LT_101_PV, and LT_101_HI. The "OBJECT PROPERTIES" list includes: Lamp_PUMP1_RUN.CurrentValue, Lamp_PUMP1_FLT.CurrentValue, Lamp_PUMP2_RUN.CurrentValue, Lamp_PUMP2_FLT.CurrentValue, Num_Flow.CurrentValue, Num_Level.CurrentValue, and five AlarmGroup1.Alarm1.HiHi.VariableName entries. Green lines indicate display bindings, and orange lines indicate alarm bindings.
- Right Sidebar:** Contains the "Properties" panel for the selected object "Lamp". It includes sections for "Geometry" (X, Y, W, H), "Data binding" (PMP_101_RUN drives CurrentValue), "General" (Type, Name, Unique id, Location, Width, Height), and "States" (Off, Text, Text color, Font, Text layout).

At the bottom, the status bar shows "HMI Copilot v0.1 | Hackathon Build | 1 screen · 19 objects · 7 tags" and the footer text "Accelerating a more sustainable and efficient world."

Press Simulate and a process model drives the screen: lamps change state, numbers move, alarms raise.

Values are driven through the project's own alarm setpoints, so every alarm it configured actually fires. Lead and standby pumps behave differently.



Built, not proposed

Two .eote files written from scratch, both open in EcoStruxure Operator Terminal Expert 4.4. Neither was ever saved by the product.

19

parts placed

7

typed tags

5

alarms

11

bindings

0

placed by hand

467

tests

474

shipped symbols indexed

50

real part types extracted

2

independent packagers
that agree

A Python reference and a TypeScript packager, structurally diffed against each other. Two implementations agreeing is a stronger claim than either passing its own tests.

Who pays, and what it costs them

	Who	Why they buy	Price
The wedge	System integrators and panel builders	Fixed-price work. Time saved is margin on hours they already quoted, and they decide fast.	₹3,499 / seat / month
Expand	Plants, utilities, OEMs	They own the house style. Standards packs make every contractor produce their HMI.	from ₹25,00,000 / year
Scale	Schneider Electric	A generative front end for the format they already ship, with no change to the product.	OEM

#	BREAK-EVEN	Indian automation engineer, fully loaded	INR 700-1,200 / hour
		Team seat	INR 41,988 / year
		Hours it must give back	41,988 / 950 = ~44 h = 5.5 days

Does generating your first draft and resolving your bindings save you five days a year? That is the whole question — and it needs no savings claim we have not measured.

The AI does not replace the HMI engineer.

It encodes the senior engineer's judgement — which part to use, which alarm every pump needs, which naming standard applies — and gives it to everyone on the team, every time.

It works today, at the file boundary, with no change to Schneider's product.

Ask us to open the generated project on your machine.

github.com/shauryaaojha/HMI_Copilot_EcoStruxure